

INSTITUTO SUPERIOR DE TRANSPORTES E COMUNICAÇÕES

Tema: Teste de Reposição — Distributed-Buy: Sistema de Gestão de Transacções de E-Commerce Distribuído, Seguro e Interoperável

Relatório Técnico

Cadeira: Sistemas Distribuídos e Paralelos

Turma: LECC31

Ano lectivo: 2026

Discente: Kenneth Dembele

Repositório: https://github.com/kennethdembele-dv/SDP-teste-de-reposicao.git

Docente: Msc. Sérgio J. Sambo

Maputo, aos 10 de Novembro de 2026

Resumo

Este relatório descreve o Distributed-Buy, um sistema de e-commerce composto por microsserviços, desenvolvido para o Teste de Reposição de Sistemas Distribuídos e Paralelos [E]. O Serviço de Inventário está escrito em Go e o Serviço de Pagamentos em Python. Comunicam por gRPC e Protocol Buffers, com mTLS e tokens JWT, e uma compra é uma saga orquestrada que devolve o stock quando o pagamento falha. O inventário usa um cluster PostgreSQL master-slave, os ficheiros partilhados ficam em NFS e o sistema corre em Docker Compose ou em Kubernetes com escalonamento automático. A API pública é REST, documentada em OpenAPI. O relatório apresenta a arquitectura com protocolos e portas, a análise de concorrência, a prova de conceito CAP e o mapeamento para a arquitectura OMA da OMG, e separa o que foi validado do que está por executar.

Índice

1. Introdução 3
2. Objectivos 3
3. Requisitos do enunciado e cobertura 3
4. Arquitectura do sistema 4
   4.1 Fluxo de uma compra 4
5. Módulo 1 — Conceitos gerais e fundamentos 5
   5.1 Redes de computadores 5
   5.2 Sistemas operativos: concorrência no inventário 5
   5.3 Linguagens de programação 5
6. Módulo 2 — Sistema distribuído vs centralizado 5
7. Módulo 3 — Transacções distribuídas: Saga 6
8. Módulo 4 — Segurança 7
   8.1 Autenticação e autorização com JWT 7
   8.2 Criptografia em trânsito com mTLS 7
9. Módulo 5 — Interoperabilidade: gRPC e Protocol Buffers 8
10. Módulo 6 — Bases de dados distribuídas 8
   10.1 Replicação master-slave 8
   10.2 Leitura descentralizada 8
   10.3 Teorema CAP 8
11. Módulo 7 — Sistemas operativos distribuídos: Kubernetes 8
12. Módulo 8 — Sistema de ficheiros distribuído 9
13. Módulo 9 — OMG e a arquitectura OMA 10
14. Módulo 10 — Serviços web distribuídos 10
15. Análise de concorrência 11
   15.1 Sem protecção (MODO_INSEGURO) 12
   15.2 Com mutex e bloqueio de linha 13
16. Prova de conceito CAP 13
17. Demonstração laboratorial e defesa 14
18. Estado da validação 15
19. Conclusão 15
Referências 15

1. Introdução

O enunciado pede que se projecte, implemente e teste um sistema de e-commerce distribuído composto por vários microsserviços, aplicando conceitos de redes, concorrência em sistemas operativos, transacções distribuídas, segurança, tolerância a falhas e interoperabilidade entre linguagens [E]. O caso central é a compra: tirar o produto do stock e cobrar o pagamento são operações em serviços e bases de dados diferentes, e o sistema tem de continuar coerente mesmo quando um serviço ou um nó falha.

2. Objectivos

- Isolar cada microsserviço num contentor, numa rede interna com DNS e controlo de portos.
- Evitar race conditions na actualização do stock com primitivas do sistema operativo.
- Comparar a arquitectura de microsserviços com uma abordagem centralizada sob carga e latência.
- Garantir a atomicidade da compra com uma saga e compensação automática.
- Autenticar com JWT e cifrar e autenticar as ligações internas com mTLS.
- Usar apenas gRPC/Protobuf na comunicação interna entre Go e Python.
- Ler os produtos de réplicas PostgreSQL e documentar o comportamento CAP numa falha.
- Executar em Kubernetes com Horizontal Pod Autoscaler e guardar ficheiros num DFS.
- Relacionar o middleware usado com a arquitectura OMA e expor uma API REST com OpenAPI.

3. Requisitos do enunciado e cobertura

Requisito [E]	Solução	Secção
M1 — Docker Compose, um contentor por serviço, rede bridge com DNS e controlo de portos	Rede loja-net 172.28.0.0/24; só o gateway publica porta (8080)	4, 5.1
M1 — Concorrência no inventário com primitivas do SO	sync.Mutex por produto + SELECT FOR UPDATE	5.2, 15
M1 — Inventário em Go, pagamentos em Python	Go 1.23 e Python 3.12	5.3
M2 — Microsserviços vs monólito; teste de carga e latência	Monólito equivalente, Locust, tc netem	6
M3 — Saga ou 2PC; compensação se o pagamento falhar	Saga orquestrada, idempotente, com recuperação	7
M4 — Servidor de identidade com JWT assinado; acesso exige token	JWT RS256 verificado em todos os serviços	8.1
M4 — TLS/mTLS entre microsserviços	CA própria; lista de serviços autorizados por método	8.2
M5 — gRPC e Protobuf entre Go e Python; sem REST interno	Contrato único proto/loja.proto	9
M6 — PostgreSQL master-slave com leitura descentralizada	1 master + 2 réplicas; leituras nas réplicas	10
M6 — Partição de rede e comportamento face a C e A	Teste com master em baixo, leitura obsoleta e partição	10.3, 16
M7 — Kubernetes local com HPA por CPU	Minikube; HPA no inventário, pagamentos e gateway	11
M8 — Upload de imagens e relatórios num DFS (MinIO ou NFS)	NFSv4 partilhado; nada no disco do contentor	12
M9 — Mapeamento gRPC/Gateway/Serviços com a OMA; gRPC vs CORBA	Tabela comparativa	13
M10 — API pública RESTful com OpenAPI; gateway converte HTTP em gRPC	FastAPI, Swagger em /docs	14
Relatório — arquitectura com protocolos e portos	Figura 1 e Tabela de portos	4
Relatório — análise de concorrência com printscreens e logs	Testes seguro e inseguro	15
Relatório — prova de conceito CAP	Discussão e resultados	16
Relatório — tabela OMA	Tabela 13	13

Requisito [E]	Solução	Secção
Demonstração — derrubar um nó de BD e continuar a ler	Guião de demonstração	17

4. Arquitectura do sistema

Cada componente corre no seu contentor e é chamado pelo nome DNS (por exemplo, `inventario:50052`), sem conhecer o endereço IP nem o número de instâncias. O cliente só fala com o API Gateway. A Figura 1 mostra os serviços, os protocolos e as portas.

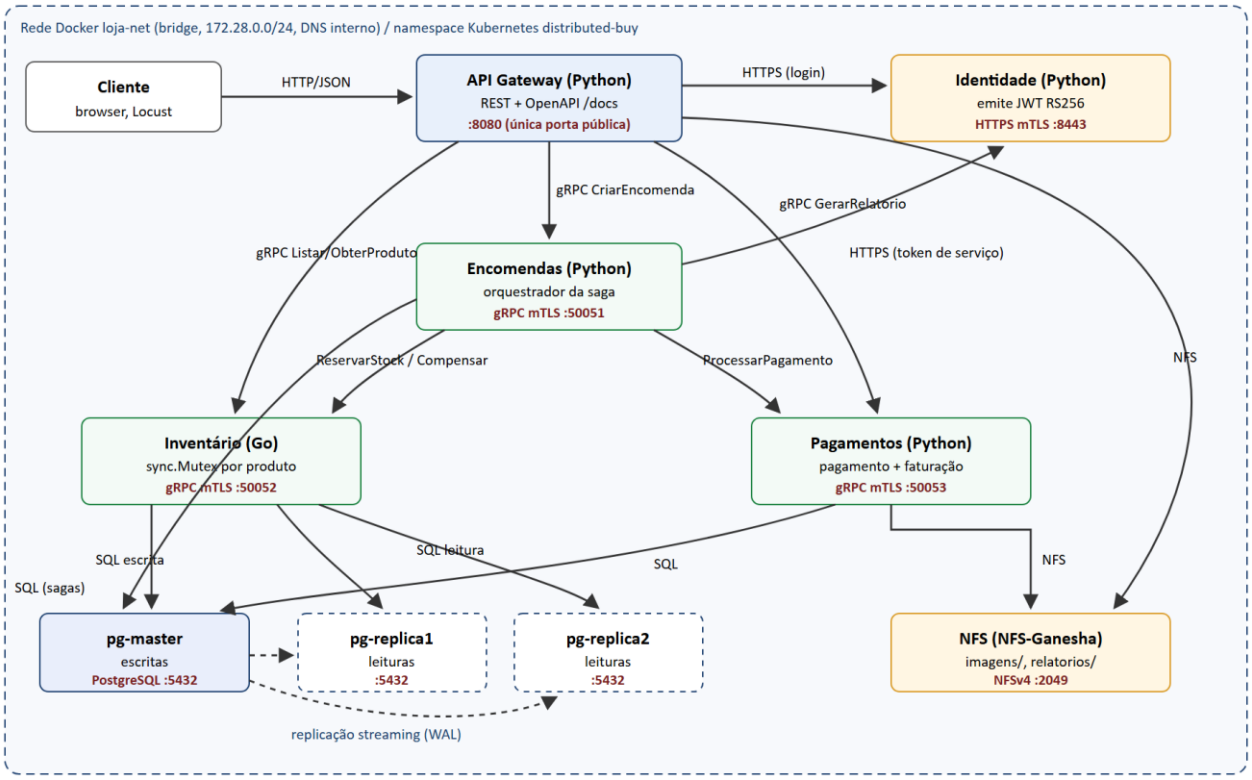


Figura 1 — Arquitectura do Distributed-Buy, com protocolos e portos.

Componente	Linguagem / tecnologia	Porto	Protocolo	Publicado
gateway	Python, FastAPI	8080	HTTP/JSON (REST, OpenAPI)	sim
identidade	Python, FastAPI	8443	HTTPS com mTLS	não
encomendas	Python, grpcio	50051	gRPC sobre HTTP/2, mTLS	não
inventario	Go, grpc-go, pgx	50052	gRPC sobre HTTP/2, mTLS	não
pagamentos	Python, grpcio	50053	gRPC sobre HTTP/2, mTLS	não
pg-master, pg-replica1, pg-replica2	PostgreSQL 16	5432	protocolo PostgreSQL; replicação WAL	não
nfs	NFS-Ganesha	2049	NFSv4 sobre TCP	sim (montagem)
monolito, pg-monolito	Python; PostgreSQL	8090	HTTP/JSON (só na comparação)	sim

4.1 Fluxo de uma compra

1. O cliente obtém um JWT em `POST /api/v1/auth/token`; o gateway pede-o à identidade por HTTPS com mTLS.
2. O cliente envia `POST /api/v1/encomendas`. O gateway verifica o token e chama `CriarEncomenda` por gRPC.
3. O orquestrador regista a saga e chama `ReservarStock` no inventário, que retira o stock.
4. O orquestrador chama `ProcessarPagamento`. Se for aprovado, a saga termina CONCLUIDA; se for recusado, chama `CompensarReserva` e o stock é devolvido (COMPENSADA).

5. O gateway converte a resposta gRPC em JSON e devolve-a com o histórico dos passos.

5. Módulo 1 — Conceitos gerais e fundamentos

5.1 Redes de computadores

O `docker-compose.yml` cria a rede `loja-net` (bridge, sub-rede 172.28.0.0/24). O DNS interno do Docker resolve os nomes dos serviços. O controlo de portos é feito com `expose` para os serviços internos e `ports` apenas para o gateway, a montagem NFS e as ferramentas de teste: as portas gRPC e PostgreSQL não são alcançáveis a partir do anfitrião.

5.2 Sistemas operativos: concorrência no inventário

Vários pedidos ao inventário correm em simultâneo, cada um numa goroutine. Sem protecção, duas compras do último artigo podiam ler ambas stock 1 e escrever ambas 0, vendendo duas unidades. O inventário usa um `sync.Mutex` por produto. `Lock` bloqueia a goroutine até o mutex estar livre [9], e quando uma compra envolve vários produtos os mutexes são obtidos sempre pela mesma ordem, o que evita deadlocks. Dentro do mutex, a transacção usa `SELECT ... FOR UPDATE`, que bloqueia a linha para outras transacções que a queiram alterar até ao fim da transacção [10]. O mutex protege as goroutines do mesmo processo; o bloqueio de linha protege também entre processos, o que é necessário quando o HPA cria várias instâncias do inventário (secção 11).

```
libertar := s.bloqueios.BloquearVarios(ids)   // sync.Mutex por produto, por ordem fixa
defer libertar()
tx, _ := s.bd.mestre.Begin(ctx)
tx.QueryRow(ctx, "SELECT stock, preco_centavos FROM produtos WHERE id = $1 FOR UPDATE", id)
if stock < q { return FailedPrecondition }
tx.Exec(ctx, "UPDATE produtos SET stock = $1 WHERE id = $2", stock-q, id)
```

Para a análise de concorrência, o inventário tem um modo de demonstração (`MODO_INSEGURO=true`) que faz a mesma leitura e escrita sem mutex nem bloqueio de linha. Os resultados dos dois modos estão na secção 15.

5.3 Linguagens de programação

	Go (Inventário)	Python (Pagamentos, Encomendas, Gateway, Identidade)
Execução	Compilada para binário nativo	Interpretada
Tipagem	Estática, verificada na compilação	Dinâmica, verificada em execução
Concorrência	Goroutines e sync.Mutex	Threads do servidor gRPC/uvicorn
Ligação entre as duas	Stubs gerados de loja.proto	Stubs gerados de loja.proto

6. Módulo 2 — Sistema distribuído vs centralizado

A versão em microsserviços foi comparada com um monólito que faz o mesmo (`servicos/monolito`): um só processo e uma só base de dados, em que a compra é uma transacção local. Os dois sistemas recebem o mesmo perfil de carga do Locust (`testes/carga/locustfile.py`): três consultas de produtos por cada compra, 50 utilizadores durante 60 segundos. A seguir repete-se o teste distribuído com 20 ms de latência de rede acrescentados com `tc netem` aos contentores do inventário, dos pagamentos e das encomendas.

Aspecto	Monólito (centralizado)	Microsserviços (distribuído)
Compra	Uma transacção local	Saga com 3 serviços e 2 bases de dados lógicas
Latência por pedido	Sem saltos de rede internos	Gateway → encomendas → inventário → pagamentos, com TLS
Falhas	Uma falha pára tudo	Falhas parciais: as leituras continuam sem o master
Escala	Escala o bloco inteiro	Cada serviço escala sozinho (HPA)
Tecnologia	Uma linguagem	Go e Python, ligados por gRPC

Cenário	Pedido	Pedidos	Falhas	Mediana (ms)	p95 (ms)	Pedidos/s
Monolito (centralizado)	GET /produtos	6897	0	5	11	116.3
Monolito (centralizado)	POST /encomendas	2405	0	6	14	40.6
Monolito (centralizado)	Agregado	9302	0	5	12	156.9
Microsservicos	GET /produtos	5909	0	12	84	99.0
Microsservicos	POST /encomendas	2010	0	39	310	33.7
Microsservicos	Agregado	7969	0	17	150	133.5
Microsservicos + 20 ms de rede	GET /produtos	4765	0	80	120	80.1
Microsservicos + 20 ms de rede	POST /encomendas	1653	0	210.0	300	27.8
Microsservicos + 20 ms de rede	Agregado	6468	0	90	250	108.7

Tabela 6 — Resultados reais do Locust: 50 utilizadores, 60 s por cenário (docs/evidencias/carga.txt). Execução em modo nativo, num só computador.

Análise. O monólito responde em 5–6 ms de mediana. Os microsserviços sobem para 12 ms nas leituras e 39 ms nas compras, porque cada compra passa por quatro chamadas gRPC com TLS e verificação de JWT, e por várias transacções em bases de dados diferentes. Com 20 ms de latência na rede, a compra mediana passa para 210 ms e o débito total cai de 134 para 109 pedidos/s: o custo da distribuição cresce com o número de saltos de rede de cada operação. Nenhum pedido falhou. O login (excluído da tabela) demora cerca de 2,5 s sob carga porque a identidade usa PBKDF2 com 120 000 iterações, um custo propositado contra ataques de força bruta. Em troca da latência, o sistema distribuído isola falhas (secção 16) e escala cada serviço separadamente (secção 11).

7. Módulo 3 — Transacções distribuídas: Saga

Uma compra altera duas bases de dados de serviços diferentes. Foi escolhido o padrão Saga orquestrado: uma saga é uma sequência coordenada de transacções locais, e quando um passo falha, os passos já concluídos são desfeitos por transacções de compensação [1]. No modelo orquestrado, um orquestrador diz aos participantes que transacção local executar [1]; aqui é o Serviço de Encomendas. Preferiu-se a saga ao 2PC porque não mantém bloqueios entre serviços enquanto espera pelo pagamento.

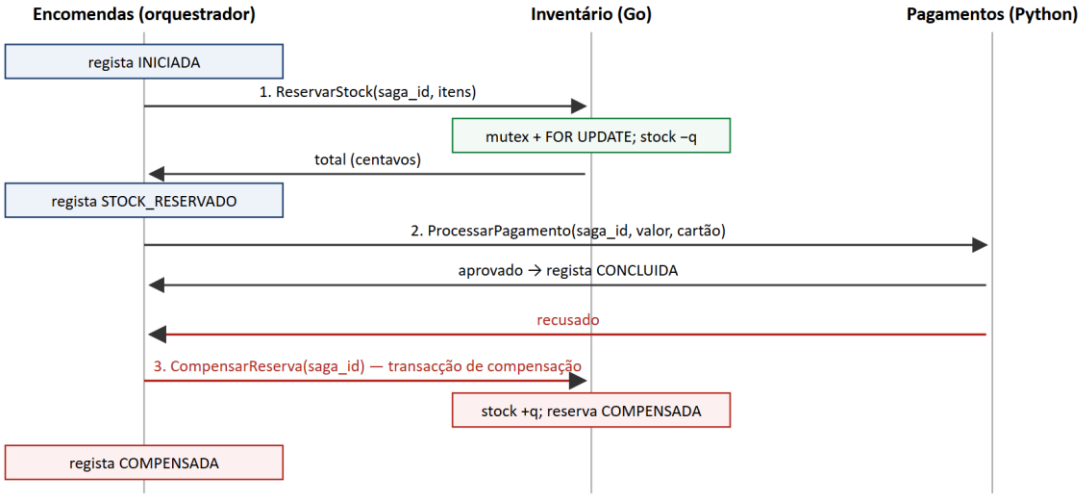


Figura 2 — Saga da compra: caminho normal e compensação.

Situação	Comportamento	Estado final
Pagamento aprovado	Fim da saga	CONCLUIDA
Pagamento recusado	CompensarReserva devolve o stock	COMPENSADA
Stock insuficiente ou produto inexistente	Nada foi retirado; não há compensação	REJEITADA

Situação	Comportamento	Estado final
Inventário sem resposta na reserva	Compensação, que é idempotente e segura mesmo sem reserva	COMPENSADA
Pagamentos sem resposta	Não compensa às cegas: o pagamento pode ter sido feito	PAGAMENTO_PENDENTE
Orquestrador cai a meio	A recuperação (de 10 em 10 s) retoma a partir do último passo gravado	—

Todas as chamadas são idempotentes pelo `saga_id`: repetir uma reserva, um pagamento ou uma compensação não os aplica duas vezes. O cliente pode ainda enviar `Idempotency-Key`, para que um POST repetido devolva a mesma encomenda. Cada passo fica gravado na tabela `saga_passos`, que serve de prova de consistência.

8. Módulo 4 — Segurança

8.1 Autenticação e autorização com JWT

O Serviço de Identidade emite tokens JWT assinados em RS256 com a chave privada, e todos os serviços os verificam com a chave pública. Valida-se a assinatura, o emissor (`iss`), o destinatário (`aud`) e a validade (`exp`) [5], além do papel (`cliente`, `admin` ou `servico`). O gateway verifica o token antes de encaminhar e passa-o ao serviço seguinte, que o verifica outra vez. Para as chamadas que faz por conta própria, como a recuperação, o orquestrador usa um token de serviço.

8.2 Criptografia em trânsito com mTLS

A comunicação interna usa TLS com autenticação mútua: o cliente também apresenta certificado, e o gRPC aceita tokens enviados nos metadados por cima do canal TLS [2]. Cada serviço tem um certificado assinado pela CA do projecto, cujo nome (CN) o identifica. Cada método tem uma lista de serviços autorizados, o que garante que apenas os nós autorizados comunicam entre si.

Método	Serviços autorizados (mTLS)	Papéis autorizados (JWT)
Inventario.Listar/ObterProduto	gateway	cliente, admin
Inventario.DefinirStock/DefinirImagem	gateway	admin
Inventario.ReservarStock/CompensarReserva	encomendas	servico
Pagamentos.ProcessarPagamento	encomendas	servico
Pagamentos.GerarRelatorioFaturacao	gateway	admin
Encomendas.Criar/Obter/ListarEncomendas	gateway	cliente, admin

```
ok   test_chave_idempotente_nao_duplica
ok   test_compensacao_falhada_e_retomada_pela_recuperacao
ok   test_compra_concluida
ok   test_inventario_em_baixo_compensa_de_forma_segura
ok   test_pagamento_recusado_devolve_o_stock
ok   test_pagamentos_em_baixo_fica_pendente_e_a_recuperacao_conclui
ok   test_passos_ficam_registados
ok   test_stock_insuficiente_rejeita_sem_compensar
ok   test_03_stock_insuficiente_e_rejeitado
ok   test_10_sem_token_e_recusado
ok   test_11_token_expirado_e_recusado
ok   test_12_token_assinado_por_outra_chave_e_recusado
ok   test_13_papel_errado_e_recusado
ok   test_20_servico_nao_autorizado_e_recusado
ok   test_openapi_documenta_a_api_publica
ok   test_pedido_sem_token_devolve_401
ok   test_sem_certificado_de_cliente_a_ligacao_falha
Ran 27 tests in 11.544s
OK
```

Tabela 8 — Testes de segurança e da saga executados com servidores gRPC reais. Saída real de sh testes/locais/executar.sh.

9. Módulo 5 — Interoperabilidade: gRPC e Protocol Buffers

Os contratos entre Go e Python estão num único ficheiro, `proto/loja.proto`. O Protobuf é um mecanismo neutro em relação à linguagem e à plataforma para serializar dados estruturados, e o compilador gera o código de cada linguagem a partir do `.proto` [3]. Os stubs Go são gerados no `docker build` do inventário e os stubs Python no build dos serviços Python. O fluxo de criação de encomendas usa apenas gRPC; o único HTTP/JSON é o da API pública do gateway.

```
service Inventario {
  rpc ListarProdutos(ListarProdutosPedido) returns (ListaProdutos);
  rpc ObterProduto(ProdutoId) returns (Produto);
  rpc ReservarStock(PedidoReserva) returns (RespostaReserva);
  rpc CompensarReserva(PedidoCompensacao) returns (RespostaCompensacao);
  rpc DefinirImagem(PedidoImagem) returns (Produto);
  rpc DefinirStock(PedidoStock) returns (Produto);
}

service Pagamentos {
  rpc ProcessarPagamento(PedidoPagamento) returns (RespostaPagamento);
  rpc GerarRelatorioFaturacao(PedidoRelatorio) returns (RespostaRelatorio);
}

service Encomendas {
  rpc CriarEncomenda(PedidoEncomenda) returns (Encomenda);
  rpc ObterEncomenda(EncomendaId) returns (Encomenda);
  rpc ListarEncomendas(ListarEncomendasPedido) returns (ListaEncomendas);
}
```

10. Módulo 6 — Bases de dados distribuídas

10.1 Replicação master-slave

O inventário usa um cluster PostgreSQL 16 com um master e duas réplicas (slaves) em replicação física por streaming. As réplicas são criadas com `pg_basebackup` e um slot de replicação cada uma, e aceitam apenas leituras [6].

10.2 Leitura descentralizada

No inventário, as escritas vão sempre para o master e as leituras alternam entre as réplicas. Se uma réplica falhar, tenta a outra e, por fim, o master. Cada resposta indica no campo `fonte` que servidor respondeu, o que permite ver o encaminhamento durante a demonstração.

10.3 Teorema CAP

Gilbert e Lynch demonstram que, das três propriedades desejadas num serviço distribuído — consistência, disponibilidade e tolerância a partições — não é possível garantir as três ao mesmo tempo [4]. Numa partição, o Distributed-Buy faz escolhas diferentes para leituras e escritas:

Operação	Escolha durante a partição	Consequência
Leitura de produtos	Disponibilidade (AP)	Responde a partir de uma réplica, eventualmente com dados atrasados
Compra (escrita)	Consistência (CP)	Recusada com HTTP 503 enquanto o master não responde

A replicação é assíncrona por omissão, por isso uma réplica pode mostrar um valor antigo durante algum tempo depois de uma escrita no master [6]. A secção 16 apresenta a prova de conceito.

11. Módulo 7 — Sistemas operativos distribuídos: Kubernetes

A pasta `k8s/` implanta o sistema num cluster local Minikube (`sh k8s/implantar.sh`). Cada serviço é um Deployment com um Service. A transparência de localização vem do DNS do cluster: os serviços continuam a chamar `inventario:50052` sem saber em que nó ou em quantos pods o inventário corre. O escalonamento é do Kubernetes, que coloca os pods nos nós, como um sistema operativo distribuído distribui processos.

O Horizontal Pod Autoscaler actualiza automaticamente o número de pods de um Deployment a partir de métricas que o Metrics Server fornece. A utilização de CPU é calculada em relação ao pedido (`requests`) de cada pod [7], por isso todos os Deployments declaram `resources.requests.cpu`. O HPA está configurado para o inventário (1 a 5 pods), os pagamentos e o gateway (1 a 4), com alvo de 50% de CPU. Como o inventário pode ter vários pods, o mutex de cada pod não chega: é o `SELECT ... FOR UPDATE` que mantém a consistência entre pods.

Execução. O sistema foi implantado num cluster Minikube 1.34 (Kubernetes 1.31, driver Docker, 2 CPU e 3 GB) com `sh k8s/implantar.sh`: 9 pods (master e 2 réplicas PostgreSQL, NFS e os cinco serviços), com o volume NFS montado no gateway e nos pagamentos. O `sh k8s/carga_hpa.sh` lançou 150 utilizadores Locust dentro do cluster durante 5 minutos e registou o HPA a cada 15 s:

Hora	Gateway CPU / pods	Inventário CPU / pods	Pagamentos CPU / pods
12:39:33 (início)	4% / 1	1% / 1	3% / 1
12:40:21	44% / 1	3% / 1	3% / 1
12:41:08	57% / 1	26% / 1	20% / 1
12:41:24	57% / 2 (HPA escalou)	26% / 1	20% / 1
12:42:42	60% / 2	25% / 1	14% / 1
12:43:45	31% / 2	24% / 1	3% / 1
12:45:34 (fim)	17% / 2	12% / 1	2% / 1

Tabela 11 — Evolução real do HPA (alvo 50% de CPU), extraída de docs/evidencias/hpa.txt.

Análise. Quando a CPU média do gateway passou o alvo de 50% (57%), o HPA criou um segundo pod, e a carga repartida desceu para 30–31% por pod: o novo pod entrou no Service sem nenhuma alteração nos clientes, o que mostra a transparência de localização. O gateway é o componente mais pesado, porque converte cada pedido REST em gRPC e verifica o JWT. O inventário, em Go, não passou de 26% e não precisou de escalar. Os pods novos não descem logo: o HPA espera uma janela de estabilização antes de reduzir, para não oscilar.

12. Módulo 8 — Sistema de ficheiros distribuído

As imagens dos produtos e os relatórios de faturação não ficam no disco dos contentores. Ficam num servidor NFSv4 (NFS-Ganesha). No Docker Compose, o volume `dfs` é do tipo `nfs` e é montado pelo Docker do anfitrião em `/mnt/dfs` no gateway e nos pagamentos; no Kubernetes, é um PersistentVolume NFS ReadWriteMany. Foi usado NFS em vez de MinIO porque as imagens oficiais do MinIO deixaram de estar disponíveis no Docker Hub; o enunciado admite as duas opções [E].

Ficheiro	Quem escreve	Endpoint
imagens/<produto>-<id>.png|jpg|webp	gateway (admin)	POST /api/v1/produtos/{id}/imagem
relatorios/faturacao-<data>-<hora>.csv	pagamentos (pedido do admin)	POST /api/v1/relatorios/faturacao
leitura de ambos	gateway	GET /api/v1/ficheiros/{pasta}/{nome}

Os ficheiros são escritos com nome temporário e depois renomeados, para que nenhum leitor veja um ficheiro incompleto. O servidor NFS não tem réplica: se ele falhar, as compras continuam, mas o upload e a leitura de ficheiros ficam indisponíveis até ele voltar.

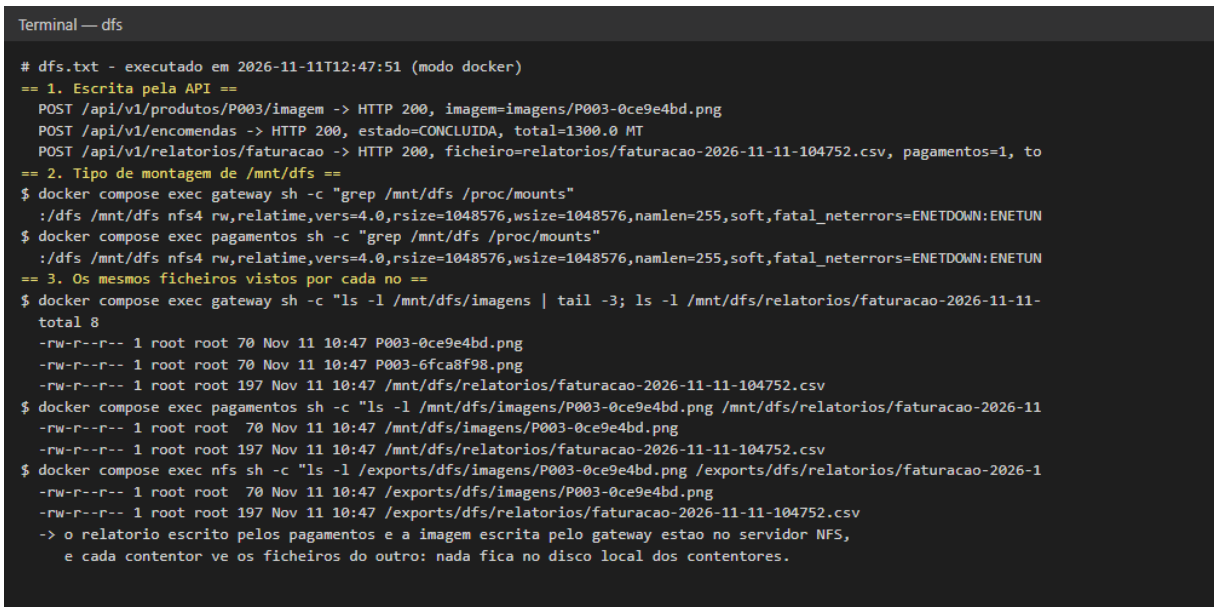


Tabela 12 — Imagem e relatório escritos pela API e vistos por todos os nós através do NFS. Captura da saída real de python testes/sistema/dfs.py (ficheiro completo em docs/evidencias/dfs.txt).

13. Módulo 9 — OMG e a arquitectura OMA

Na Object Management Architecture (OMA), o centro do modelo é o Object Request Broker (ORB), a infraestrutura que permite a objectos comunicar independentemente da plataforma. À sua volta estão os CORBA Services, as CORBA Facilities e os Application Objects, que invocam métodos de objectos remotos através do ORB [11]. No CORBA, a interface define-se em OMG IDL e o ORB encaminha a invocação pela rede até ao ORB do objecto remoto [12]. O gRPC cumpre hoje o mesmo papel: o cliente chama um método do stub como se fosse local, e a biblioteca gRPC serializa o pedido em Protobuf e envia-o por HTTP/2. Ao contrário do CORBA, o gRPC não tem referências a objectos com estado nem um Naming Service próprio: os serviços são encontrados por DNS.

Conceito OMG/OMA	CORBA	Distributed-Buy (gRPC)
Object Request Broker	ORB (por exemplo JacORB) e protocolo IIOP	Biblioteca gRPC (grpc-go, grpcio) sobre HTTP/2
Linguagem de interfaces	OMG IDL	Protocol Buffers (loja.proto)
Stub e skeleton	Gerados pelo compilador IDL	Gerados pelo protoc (Go e Python)
Application Objects	Servants de aplicação	Serviços Inventário, Pagamentos e Encomendas
CORBA Services: Naming	Naming Service	DNS do Docker/Kubernetes
CORBA Services: Security	CORBA Security Service	mTLS + JWT (interceptores gRPC)
CORBA Services: Transactions	Object Transaction Service	Saga orquestrada (Serviço de Encomendas)
CORBA Facilities	Facilidades horizontais/verticais	Identidade, relatórios de faturação
Interoperabilidade entre ORBs	IIOP	HTTP/2 + Protobuf, em qualquer linguagem suportada
Ponte para clientes externos	—	API Gateway (REST → gRPC)

14. Módulo 10 — Serviços web distribuídos

A API pública é RESTful e está descrita em OpenAPI 3.1, uma descrição normalizada e independente de linguagem para APIs HTTP [8]. O FastAPI gera a especificação (`/openapi.json`, exportada em `docs/openapi.json`) e a interface Swagger em `/docs`. O gateway é a porta de entrada: converte cada pedido HTTP/JSON numa chamada gRPC e cada código gRPC no código HTTP correspondente (por exemplo, `FAILED_PRECONDITION` em 409 e `UNAVAILABLE` em 503).

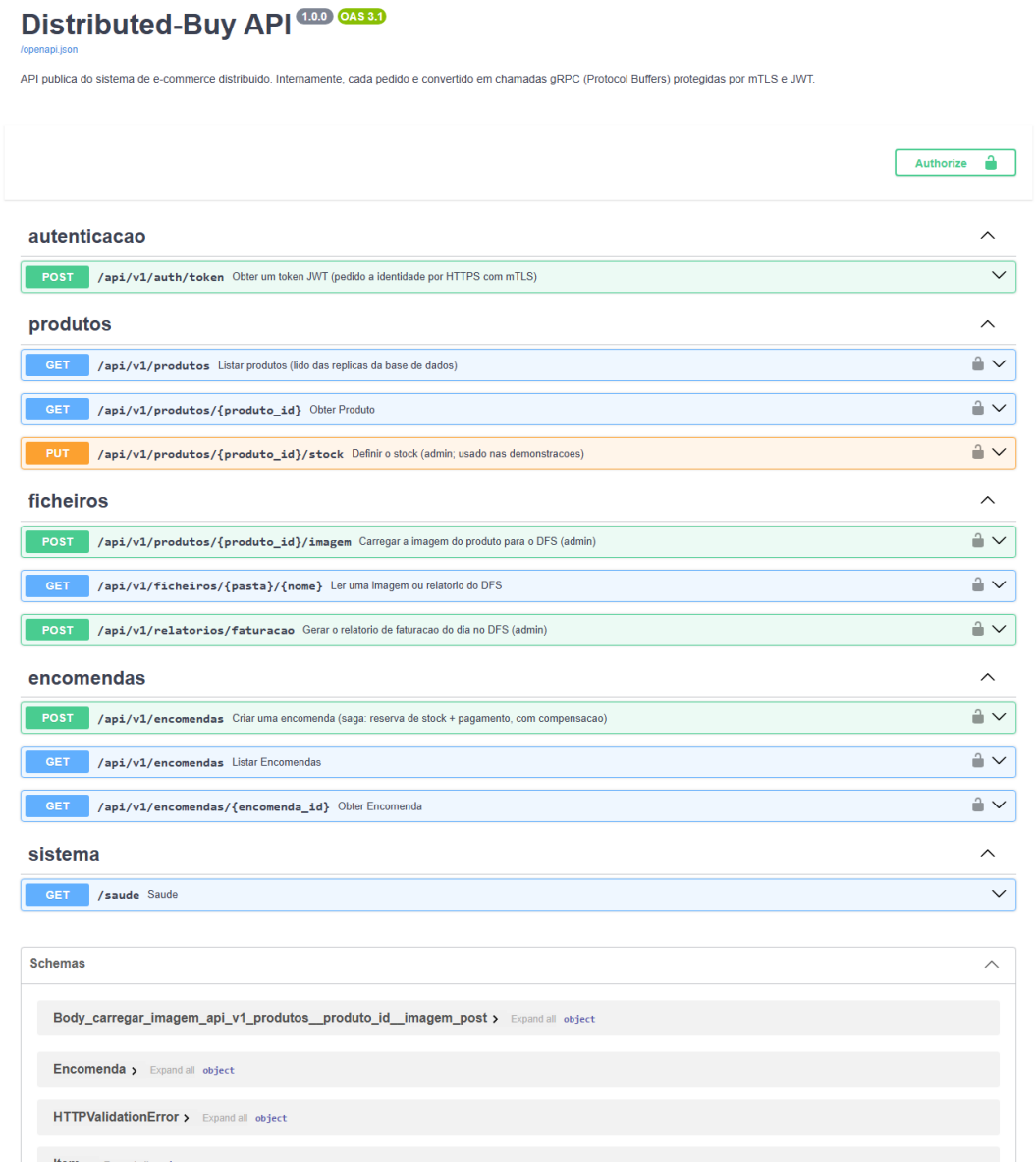


Figura 3 — Swagger UI do gateway (captura real, gateway em execução).

Além do Swagger, o gateway serve uma interface gráfica web em `/` (`servicos/gateway/web/`), feita em HTML, CSS e JavaScript sem bibliotecas externas. Usa apenas a API REST pública, como qualquer outro cliente. Tem uma loja com catálogo (mostrando a réplica que respondeu a cada leitura), carrinho e checkout com o percurso da saga, a lista de encomendas, a administração (stock, upload de imagens para o NFS e relatórios de faturação) e duas demonstrações para a defesa: compras concorrentes lançadas a partir do browser, com verificação da consistência, e um monitor de leituras para mostrar o comportamento CAP quando o master é derrubado.

15. Análise de concorrência

O teste `testes/sistema/concorrencia.py` define o stock do produto P002 em 50 unidades e envia 200 encomendas simultâneas de 1 unidade, a partir de 100 threads. Corre duas vezes: com o inventário normal e com `MODO_INSEGURO=true`. Uma compra está consistente quando cada unidade vendida corresponde a uma unidade retirada do stock.

Resultados (Docker Compose). Sem protecção, 197 das 200 encomendas foram aceites, mas o stock só desceu 15 unidades: 182 vendas não correspondem a stock retirado. Várias goroutines leram o mesmo valor e escreveram o mesmo resultado (actualizações perdidas), o que prova a race condition. Em modo nativo o resultado foi do mesmo tipo (200 vendas para 44 unidades retiradas). Com o mutex e o bloqueio de linha,

foram vendidas exactamente 50 unidades, as outras 150 encomendas foram rejeitadas por falta de stock e o stock final é 0. Os logs mostram as goroutines em fila (LOCK_ESPERA), uma de cada vez a obter o mutex (LOCK_OBTIDO), a actualizar o stock (RESERVA) e a libertá-lo (LOCK_LIBERTADO).

15.1 Sem protecção (MODO_INSEGURO)

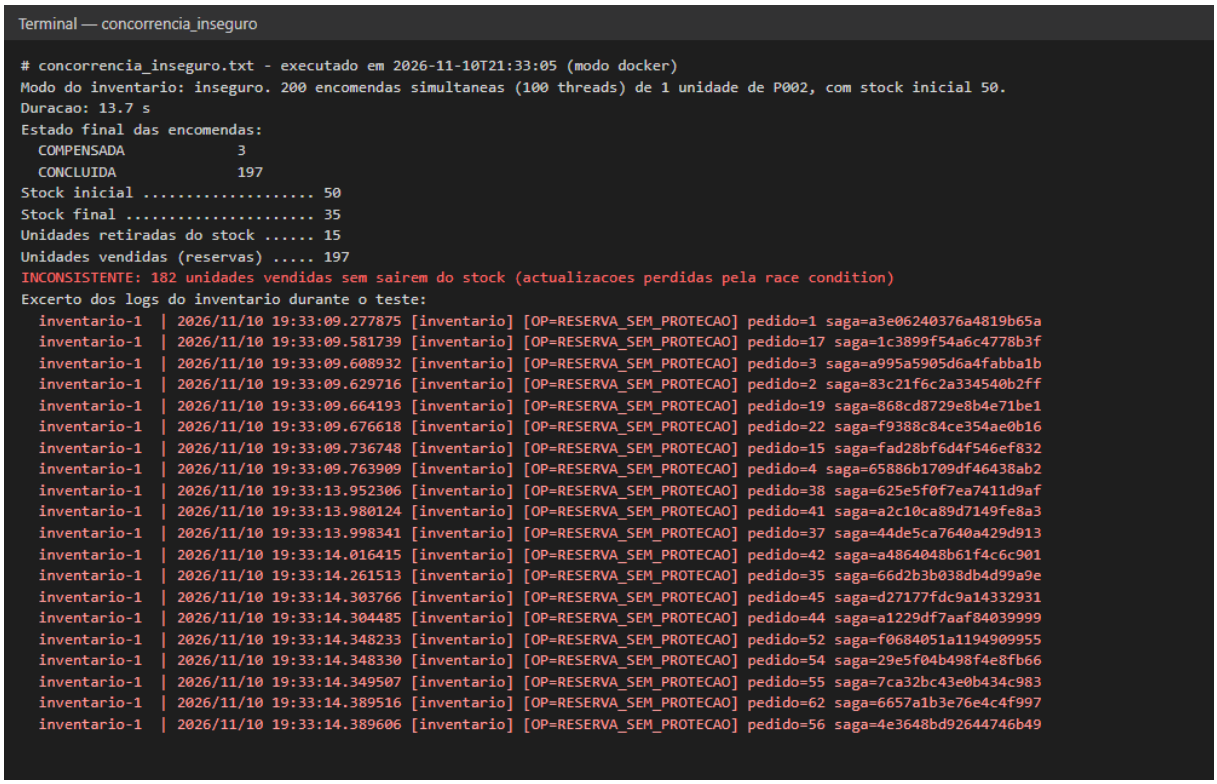


Tabela 15.1 — Race condition sem mutex. Captura da saída real de python testes/sistema/concorrencia.py inseguro (ficheiro completo em docs/evidencias/concorrencia_inseguro.txt).

15.2 Com mutex e bloqueio de linha

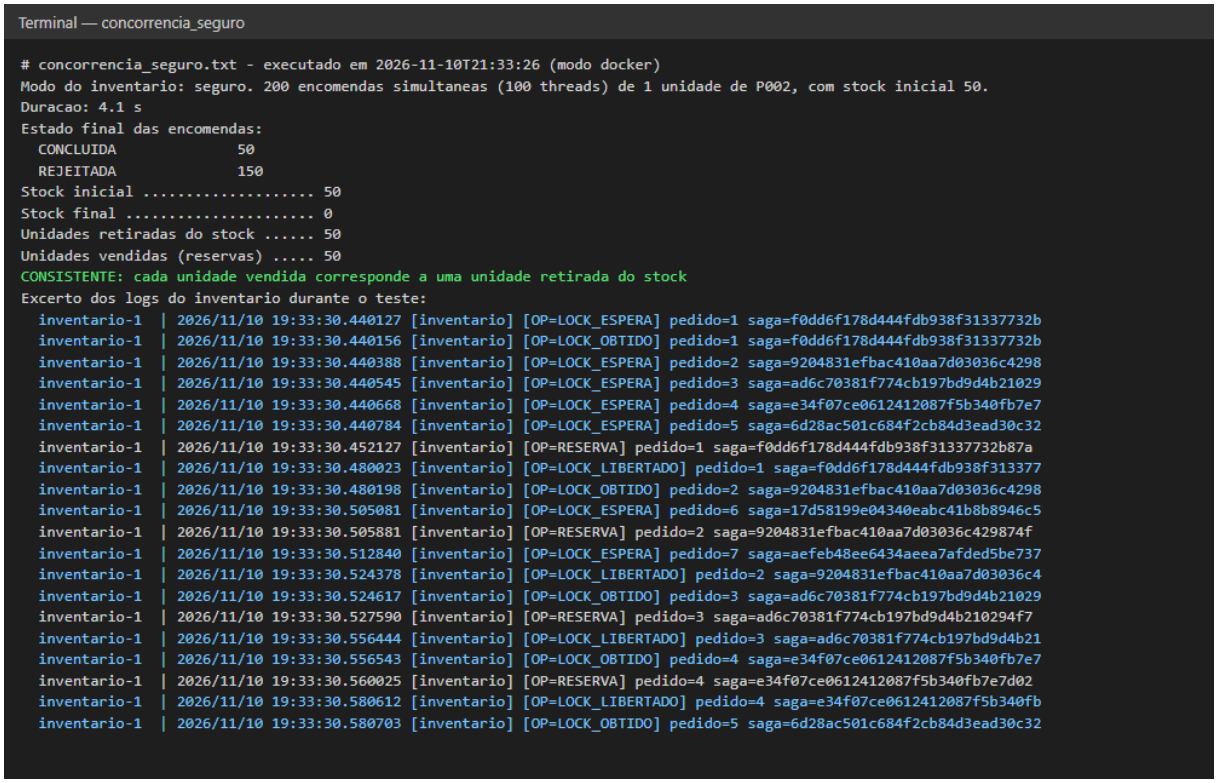


Tabela 15.2 — Compras concorrentes com protecção. Captura da saída real de python testes/sistema/concorrencia.py seguro (ficheiro completo em docs/evidencias/concorrencia_seguro.txt).

A mesma propriedade está coberta pelo teste Go `TestMutexImpedeVendaAcimaDoStock`: 500 goroutines leem e escrevem o stock com uma pausa entre as duas operações, e o teste exige exactamente 50 vendas. Passou com `go test` (Go 1.23.4); no `docker build` corre também com o detector de corridas (`-race`).

16. Prova de conceito CAP

O teste `testes/sistema/cap.py` faz quatro experiências sobre o cluster do inventário: (1) leitura normal; (2) `docker compose stop pg-master`, seguido de leituras e de uma compra; (3) `pg_wal_replay_pause()` nas duas réplicas, uma alteração do stock no master e novas leituras; (4) partição de rede com `docker network disconnect` da réplica 1.

Experiência	Comportamento esperado	Propriedade
Master em baixo	Leituras HTTP 200 com fonte replica1/replica2; compra HTTP 503	A nas leituras, C nas escritas
Replicação pausada	As réplicas devolvem o stock antigo até a replicação ser retomada	Consistência eventual
Réplica 1 isolada	LEITURA_FALHOU nos logs; as leituras passam à réplica 2	Tolerância a partições

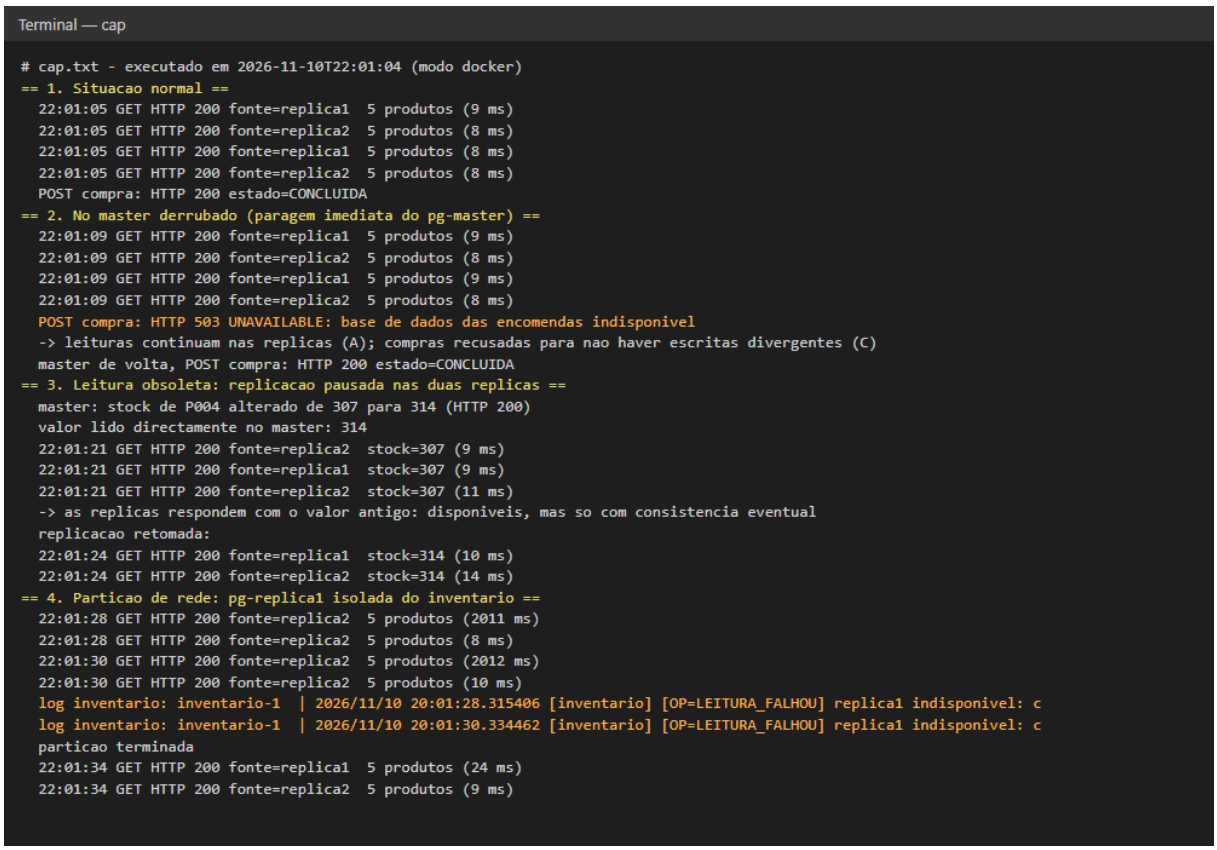


Tabela 16 — Execução da prova de conceito CAP. Captura da saída real de python testes/sistema/cap.py (ficheiro completo em docs/evidencias/cap.txt).

Esta execução foi feita em Docker Compose: o master foi derrubado com `docker compose stop pg-master` e a partição foi criada com `docker network disconnect` da réplica 1. A mesma experiência foi também repetida em modo nativo, com um proxy de rede a cortar a ligação, com o mesmo comportamento. A primeira execução em Docker revelou um defeito: com o contentor do master parado, o nome `pg-master` desaparece do DNS do Docker e o pool de ligações do Serviço de Encomendas não recuperava sozinho quando o master voltava. Foi corrigido com um pool que se recria após uma falha (`servicos/comum/pool.py`); a execução apresentada já inclui a correcção, e a compra depois do regresso do master foi aceite.

Discussão. Com o master em baixo, as leituras continuaram (HTTP 200, alternando entre as réplicas) e a compra foi recusada com HTTP 503: o sistema escolheu disponibilidade para as leituras e consistência para as escritas, para não haver duas versões do mesmo stock. Com a replicação pausada, o master tinha 314 unidades de P004, mas as réplicas continuavam a responder 307: o dado estava disponível, mas desactualizado, e convergiu assim que a replicação foi retomada. Na partição, cada leitura que calhava à réplica isolada esperou o tempo limite de 2 s (LEITURA_FALHOU) e foi respondida pela réplica 2: a tolerância a partições custa latência. Promover uma réplica a master devolveria as escritas, mas, sendo a replicação assíncrona, poderia perder as últimas alterações.

17. Demonstração laboratorial e defesa

Código fonte: https://github.com/kennethdembele-dv/SDP-teste-de-reposicao (repositório Git com o docker-compose.yml, os manifestos Kubernetes, os testes e as evidências).

O guião completo está em `docs/GUIAO_DEMONSTRACAO.md`. A demonstração pedida no enunciado é derrubar um nó de base de dados e mostrar que o sistema continua a ler os produtos [E]:

```
docker compose stop pg-master
curl -H "Authorization: Bearer $TOKEN" localhost:8080/api/v1/produtos   # 200, fonte=replica1|replica2
curl -X POST ... localhost:8080/api/v1/encomendas                          # 503: escritas suspensas
docker compose start pg-master
```

18. Estado da validação

Tudo foi executado num computador Windows 10 (4 CPU, 8 GB) com Docker Desktop 29.9.3 sobre WSL2. Antes de o Docker estar instalado, os mesmos testes foram corridos em modo nativo (`scripts/nativo.py`), com PostgreSQL 16.4 e o inventário compilado em Go 1.23.4; o teste de carga apresentado é dessa execução. As restantes evidências são da execução em Docker Compose.

Componente	Estado
Inventário em Go: go vet e go test -race (no docker build)	Executado
Saga, idempotência e recuperação (8 testes)	Executado
gRPC + mTLS + JWT entre serviços reais (13 testes, 7 recusas de segurança)	Executado
Identidade HTTPS mTLS e gateway REST/OpenAPI (6 testes, Swagger)	Executado
Docker Compose: 9 contentores, rede bridge, réplicas em streaming	Executado
NFS montado como nfs4 e partilhado entre contentores (secção 12)	Executado
Análise de concorrência, seguro e inseguro (secção 15)	Executado (Docker e nativo)
Prova de conceito CAP com master derrubado e partição (secção 16)	Executado (Docker e nativo)
Teste de carga monólito vs microsserviços vs latência (secção 6)	Executado (modo nativo)
Kubernetes e HPA (secção 11)	Executado (Minikube)

19. Conclusão

O Distributed-Buy cumpre os dez módulos com uma arquitectura simples de defender. A consistência da compra vem de duas camadas: o mutex e o bloqueio de linha, que impedem vender stock que não existe, e a saga com compensação, que impede cobrar sem stock ou reter stock sem pagamento. A segurança combina identidade dos serviços (mTLS) com identidade dos utilizadores (JWT), e o gRPC liga Go e Python com um único contrato. As principais limitações são o servidor NFS sem réplica, a promoção manual de uma réplica a master e as leituras eventualmente atrasadas durante uma partição.

Referências

[E] Enunciado do Teste de Reposição — Miniteste e Testes, Sistemas Distribuídos e Paralelos, ISUTC, 2026.

[1] Richardson, C. Pattern: Saga. microservices.io. https://microservices.io/patterns/data/saga.html

[2] gRPC Authors. Authentication. https://grpc.io/docs/guides/auth/

[3] Google. Protocol Buffers Documentation: Overview. https://protobuf.dev/overview/

[4] Gilbert, S.; Lynch, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. https://users.ece.cmu.edu/~adrian/731-sp04/readings/GL-cap.pdf

[5] Jones, M.; Bradley, J.; Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519.html

[6] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Log-Shipping Standby Servers. https://www.postgresql.org/docs/16/warm-standby.html

[7] Kubernetes. Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[8] OpenAPI Initiative (2021). OpenAPI Specification v3.1.0. https://spec.openapis.org/oas/v3.1.0

[9] The Go Authors. sync package: Mutex. https://pkg.go.dev/sync#Mutex

[10] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Explicit Locking (Row-Level Locks). https://www.postgresql.org/docs/16/explicit-locking.html

[11] Object Management Group. Object Management Architecture. https://www.omg.org/oma/

[12] Object Management Group. CORBA FAQ; Common Object Request Broker Architecture 3.4 (2021). https://www.omg.org/corba/faq.htm ; https://www.omg.org/spec/CORBA/